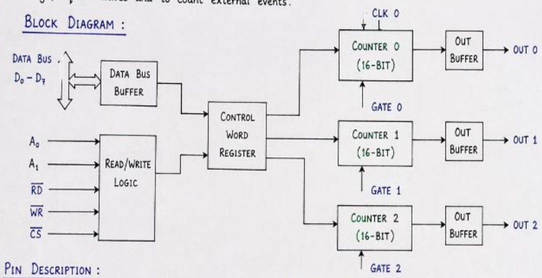
PROGRAMMABLE INTERVAL TIMER (8253/8254)

The 8253 (8254) is a programmable interval timer. It is used to generate accurate time delays, square waves and to count external events.

BLOCK DIAGRAM :



FUNCTIONING :

1. A control word is written into the control word register to set the mode (0-5), read/write format and which counter is to be used.
2. The count value (16-bit) is written into the selected counter.
3. The counter starts counting on the incoming clock pulses, if the gate input is active.
4. When the count reaches zero, the output (OUT) changes state (depending on the mode) and the counter either stops or reloads automatically.

PIN DESCRIPTION :

- $D_0 - D_7$: Bidirectional data bus.
- A_0, A_1 : Address inputs to select counter or control word.
- $\overline{RD}$: Read strobe (active low).
- $\overline{WR}$: Write strobe (active low).
- $\overline{CS}$: Chip select (active low).
- CLK 0,1,2 : Clock inputs for counter 0, 1 and 2.
- GATE 0,1,2 : Gate inputs for counter 0, 1 and 2.
- OUT 0,1,2 : Output of counter 0, 1 and 2.

INTERIOR ORGANIZATION :

- It contains three independent 16-bit counters (Counter 0, Counter 1, Counter 2).
- Each counter can be programmed by the control word register.
- The data bus buffer isolates the internal bus from the system data bus.
- Read/Write logic, with A_0 and A_1, selects either a counter or the control word register.

FEATURES :

- Three independent 16-bit counters.
- Six programmable modes (0 to 5).
- Binary counting (up to 65536).
- Can generate time delay, square waves, rate generator, event counter etc.
- 8254 is an advanced version of 8253 (operates at higher clock frequency).

MODES OF OPERATION (0 to 5) :

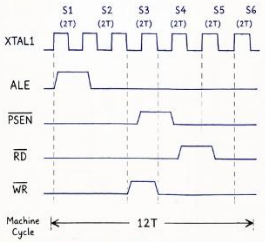
Mode	Function	Mode	Function
0	Interrupt on terminal count	3	Square wave generator
1	Hardware retriggerable one-shot	4	Software triggered strobe
2	Rate generator	5	Hardware triggered strobe

APPLICATIONS : Time delay generation, Frequency division, Event counting, Digital clock, Data acquisition systems, etc.

8051 MICROCONTROLLER

TIMING DIAGRAM

The 8051 uses a crystal oscillator. One machine cycle consists of 12 oscillator periods.



STATES OF MACHINE CYCLE

- S1 (2T) : The low-order address is placed on Port 0.
- S2 (2T) : The high-order address is placed on Port 2. ALE goes high to latch the address.
- S3 (2T) : The actual read or write operation takes place. $\overline{PSEN}$, $\overline{RD}$ or $\overline{WR}$ goes low.
- S4 (2T) : Data is valid (for read) or data is written (for write).
- S5 (2T) : The signals return to inactive state.
- S6 (2T) : The bus is free for next operation.

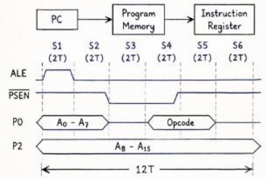
Note : T = Time period of one oscillator cycle. One machine cycle = 12T.

EXECUTION CYCLES OF 8051

8051 performs all operations by executing instructions. Each instruction takes one or more machine cycles. The main execution cycles are :

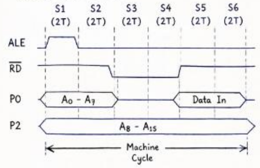
1. FETCH CYCLE

Used to fetch opcode from program memory.



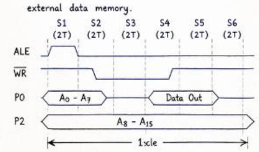
2. MEMORY READ CYCLE

Used when data is read from internal or external data memory.



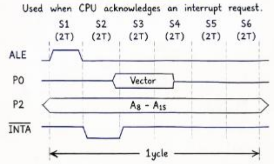
3. MEMORY WRITE CYCLE

Used when data is written to internal or external data memory.



4. INTERRUPT ACKNOWLEDGE CYCLE

Used when CPU acknowledges an interrupt request.



SUMMARY TABLE

Cycle	Used For	Active Signal	Data on P0
Fetch Cycle	Fetch opcode from program memory	$\overline{PSEN}$	Opcode
Memory Read Cycle	Read data from memory	$\overline{RD}$	Data In
Memory Write Cycle	Write data to memory	$\overline{WR}$	Data Out
Interrupt Acknowledge Cycle	Acknowledge interrupt	$\overline{INTA}$	Interrupt Vector

IMPORTANT POINTS

- Each machine cycle = 12 oscillator periods.
- Each state = 2 oscillator periods.
- All instructions are executed by one or more machine cycles.
- Timing depends on the crystal frequency.

MEMORY INTERFACING IN 8085

Introduction :

Memory interfacing in 8085 means connecting external memory (RAM / ROM) with the microprocessor so that it can store instructions, data and results.

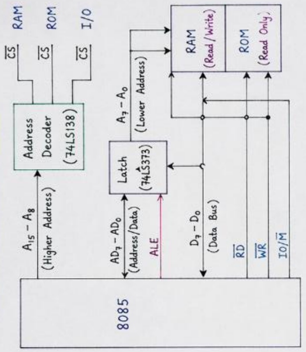
8085 Memory Organization :

- 8085 has 16-bit address bus.
- It can address $2^{16} = 65536 = 64$ KB memory locations.
- Memory is byte organized.
- Lower 8-bit ($AD_0 - AD_7$) are multiplexed address/data bus.

Required Signals :

1. Address bus : $A_0 - A_{15}$ (16 lines)
2. Data bus : $D_0 - D_7$ (8 lines)
3. Control signals :
   - $\overline{RD}$ → Read signal (from 8085)
   - $\overline{WR}$ → Write signal (from 8085)
   - $IO/\overline{M}$ → 0 for memory operation (1 for I/O operation)
   - ALE → Address Latch Enable (used for demultiplexing $AD_0 - AD_7$)

Basic Interfacing Block Diagram :



Working :

1. 8085 places 16-bit address on the address bus.
2. Higher address lines ($A_{15} - A_8$) are decoded by decoder.
3. Decoder selects the required memory chip by making its $\overline{CS}$ active (low).
4. Lower address lines ($A_7 - A_0$) are latched using 74LS373 when ALE = 1.
5. For Read operation → $\overline{RD} = 0$, data is read from memory. For Write operation → $\overline{WR} = 0$, data is written into memory.

Types of Memory Interfacing :

1. RAM Interfacing → Both Read and Write operations.
2. ROM Interfacing → Only Read operation.

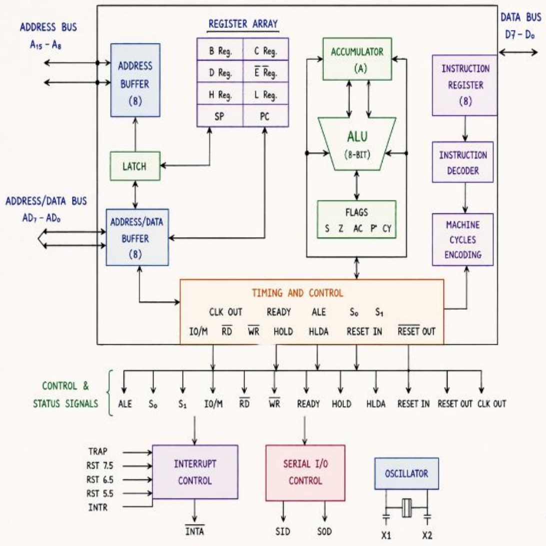
Address Decoding Methods :

1. Full (Absolute) Decoding → Uses all higher address lines. Each memory has unique address.
2. Partial Decoding → Uses some higher address lines. Simpler but may cause address overlap.

Key Points :

- 8085 can address 64 KB memory.
- $AD_0 - AD_7$ are multiplexed, demultiplexed using ALE and latch.
- Proper decoding ensures that only one memory chip is selected at a time.
- $\overline{RD}$ and $\overline{WR}$ control the data transfer.

MICROPROCESSOR 8085 - BLOCK DIAGRAM AND EXPLANATION



EXPLANATION (in brief)

1. ALU (Arithmetic Logic Unit)
   Performs arithmetic and logical operations such as addition, subtraction, AND, OR, compare, increment, etc.
2. Accumulator (A)
   8-bit register used to store data and intermediate results of ALU operations.
3. Register Array
   Consists of six 8-bit general purpose registers (B, C, D, E, H, L), Stack Pointer (SP) and Program Counter (PC).
4. Flags Register
   Contains five flags: Sign (S), Zero (Z), Auxiliary Carry (AC), Parity (P) and Carry (CY).
5. Instruction Register & Decoder
   Holds the instruction code and decodes it to generate the sequence of micro-operations.
6. Timing and Control
   Generates timing signals and control signals to coordinate all operations of the system.
7. Address/Data Buffer, Latch and Address Buffer
   $AD_7 - AD_0$ is multiplexed address/data bus. ALE signal latches the lower address and $A_{15} - A_8$ are provided through address buffer.
8. Interrupt Control
   Handles interrupt requests of various types and sends INTA (Interrupt Acknowledge) signal to the peripheral.
9. Serial I/O Control
   Provides serial communication through SID (Serial Input Data) and SOD (Serial Output Data).
10. Oscillator
    Provides clock pulses to the 8085 using external crystal and capacitors.

8085 Program to Add Two Bytes

Problem :

Write an 8085 program to add two 8-bit numbers (bytes) and store the result.

Assumption :

Let the two bytes are stored in memory locations 2000H and 2001H. The result (sum) will be stored in 2002H.

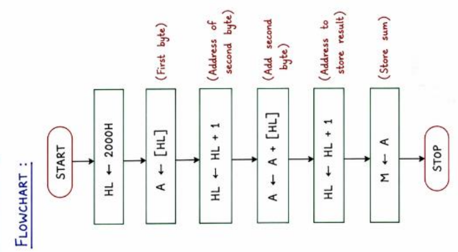
Program :

Address	Label	Instruction	Comment
8000H		LXI H, 2000H	HL ← 2000H
8003H		MVI A, M	A ← [2000H] (First byte)
8004H		INX H	HL ← HL + 1 (HL = 2001H)
8005H		ADD M	A ← A + [2001H] (Add second byte)
8006H		INX H	HL ← HL + 1 (HL = 2002H)
8007H		MOV M, A	Store result in [2002H]
8008H		HLT	Stop

Explanation :

- LXI H, 2000H → Load address of first byte in HL pair.
- MVI A, M → Load first byte into accumulator A.
- INX H → Point HL to second byte.
- ADD M → Add second byte to A.
- INX H → Point HL to next location for result.
- MOV M, A → Store the sum in memory.
- HLT → Stop the program.

Flowchart :



Note :

- ADD instruction adds the memory byte to accumulator A.
- Result is stored in 2002H.

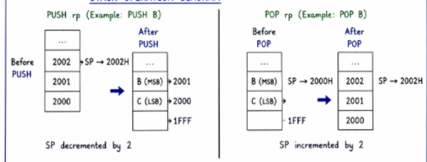
8085 Instruction Set : Stack and Machine Control Groups

Stack Group

These instructions are used to store the contents of register or Program Counter in the stack and to retrieve the data from stack.

Instruction	Mnemonic	Operation	Bytes	Cycles
Push register pair	PUSH rp	SP ← SP − 2 (MSB) → (SP + 1) (LSB) → (SP)	1	11
Pop register pair	POP rp	(SP) → (LSB) (SP + 1) → (MSB) SP ← SP + 2	1	10
Push PSW	PUSH PSW	Push Accumulator and Flags in stack	1	11
Pop PSW	POP PSW	Pop data from stack into Accumulator and Flags	1	10
Exchange top of stack with HL	XTHL	(SP) ↔ L (SP + 1) ↔ H	1	18
Exchange top of stack with DE	XCHG	H ↔ D L ↔ E	1	4
Load SP to HL	SPHL	(SP) → L (SP + 1) → H	1	5
Load HL to SP	PCHL	L → (SP) H → (SP + 1)	1	5

Stack Operation Diagram



Note : SP → Stack Pointer, MSB → Most Significant Byte, LSB → Least Significant Byte

Machine Control Group

These instructions are used to control the sequence of program execution and to control interrupts.

Instruction	Mnemonic	Operation	Bytes	Cycles
No Operation	NOP	No operation is performed	1	4
Halt	HLT	Processor is halted until an interrupt or reset occurs	1	7
Stop	STC	Processor is stopped. It can be restarted only by reset	1	4
Enable Interrupts	EI	Enables all maskable interrupts (IE flip-flop = 1)	1	4
Disable Interrupts	DI	Disables all maskable interrupts (IE flip-flop = 0)	1	4
Restart n	RST n	Call subroutine at fixed memory location n × 8 (n = 0 to 7)	1	11
Return	RET	Return from subroutine. PCL ← (SP), SP ← SP + 1 PCH ← (SP), SP ← SP + 1	1	10
Return from Interrupt	RIM	Read interrupt mask and status into Accumulator	1	10
Set Interrupt Mask	SIM	Set interrupt mask (IMR) and control serial output and interrupts	1	10

RST n Vector Addresses

n	0	1	2	3	4	5	6	7
Address	0000H	0008H	0010H	0018H	0020H	0028H	0030H	0038H

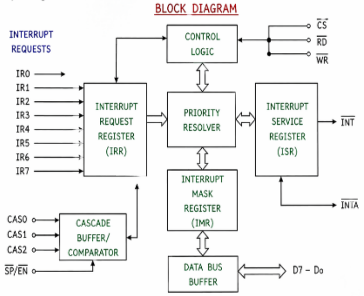
Note :

1. All instructions are of 1 byte except where operand is needed.
2. Cycles shown are for 8085.

Programmable Interrupt Controller 8259A

The 8259A is a programmable interrupt controller used to manage multiple interrupt requests for the microprocessor. It can handle 8 interrupt inputs and has several operating modes.

Block Diagram



Organization (Main Blocks)

- Interrupt Request Register (IRR) : Stores all incoming interrupt requests.
- Priority Resolver : Determines the highest priority request.
- Interrupt Service Register (ISR) : Keeps track of interrupts currently being serviced.
- Interrupt Mask Register (IMR) : Masks (disables) selected interrupts.
- Control Logic : Manages read/write operations, command decoding, interrupt acknowledge and mode control.
- Data Bus Buffer : 8-bit tri-state buffer for data bus exchange.
- Cascade Buffer/Comparator : Used in buffered or cascaded systems.

Pin Description (Important)

IR0 – IR7	:	Interrupt request inputs (8 lines)
$\overline{INT}$	:	Interrupt output to CPU
$\overline{INTA}$	:	Interrupt acknowledge input from CPU
D7 – Do	:	Data bus (tri-state, bidirectional)
$\overline{CS}$	:	Chip select (active low)
$\overline{RD}$	:	Read control input (active low)
$\overline{WR}$	:	Write control input (active low)
A0	:	Address input (0 – command, 1 – data)
$\overline{SP}/\overline{EN}$	:	Slave program / Enable buffer
$\overline{CAS0}$–$\overline{CAS2}$	:	Cascade lines for connecting multiple 8259A

Working

- When any device requests an interrupt through IR0–IR7, the request is latched in the Interrupt Request Register (IRR).
- The Priority Resolver checks the priority of all pending requests and selects the highest priority request that is not masked in IMR.
- The selected request is moved from IRR to ISR and an interrupt signal ($\overline{INT}$) is sent to the CPU.
- CPU responds with an Interrupt Acknowledge pulse ($\overline{INTA}$).
- On $\overline{INTA}$, the 8259A places the interrupt type (vector) on the data bus.
- After the service routine is completed, CPU sends End of Interrupt (EOI) command to 8259A and the corresponding bit in ISR is cleared.
- New requests can then be serviced.

Priority Structure (Default)

Highest Priority → Lowest Priority

IR0	IR1	IR2	IR3	IR4	IR5	IR6	IR7
(7)	(6)	(5)	(4)	(3)	(2)	(1)	(0)

IR0 has highest priority and IR7 has lowest priority in fully nested mode.

Modes of Operation

1. Fully Nested Mode
2. Automatic EOI Mode
3. Special Mask Mode
4. Special Fully Nested Mode
5. Buffered Mode

8051 Instructions : RIM and SIM

RIM (Read Interrupt Mask)

Function :

RIM copies the contents of the Interrupt Mask Register (IMR) and the pins status of the 8051 into the Accumulator (A).

Operation :

After execution of RIM instruction, the Accumulator is loaded as shown below.

Accumulator (A)

D7	D6	D5	D4	D3	D2	D1	D0
SID (Serial Input Data)	SOD (Serial Output Data)	X (Not used)	RS1 (Serial Port Control bit 1)	RS0 (Serial Port Control bit 0)	P2.7	P2.6	P2.5

- SID : Serial input data (from pin P3.7)
- SOD : Serial output data (to pin P3.7)
- RS1, RS0 : Serial port control bits
- P2.7, P2.6, P2.5 : Status of interrupt request pins
  - P2.7 → RI (Serial Port interrupt)
  - P2.6 → TI (Serial Port interrupt)
  - P2.5 → IE0 (External interrupt 0)

8051 Pin Status

Pin	Function
P2.7	RI (Serial Port Interrupt)
P2.6	TI (Serial Port Interrupt)
P2.5	IE0 (External Interrupt 0)

Note :

RIM does not affect any flag.

SIM (Set Interrupt Mask)

Function :

SIM loads the Accumulator contents into the Interrupt Mask Register (IMR) and also controls the Serial Port and interrupts.

Operation :

The Accumulator is arranged as shown below before executing SIM instruction.

Accumulator (A)

D7	D6	D5	D4	D3	D2	D1	D0
SOD (Serial Output Data)	SDE (Serial Data Enable)	X (Not used)	R7.5 (Reset Flag)	MSE (Mask Set Enable)	M7.5 (Mask bit for R7.5)	M6.5 (Mask bit for R6.5)	M5.5 (Mask bit for R5.5)

- SOD : Data to be sent on serial port pin P3.7.
- SDE : Serial Data Enable.
  - 1 = Enables serial port transmission of SOD.
  - 0 = Disables transmission.
- R7.5 : Reset flag for external interrupt 7.5.
  - 1 = Clears the interrupt request flag.
  - 0 = No effect.
- MSE : Mask Set Enable.
  - 1 = The mask bits (M7.5, M6.5, M5.5) are updated into IMR.
  - 0 = No change in IMR.
- M7.5, M6.5, M5.5 : Mask bits for external interrupts 7.5, 6.5 and 5.5.
  - 1 = Mask (Disable) the interrupt.
  - 0 = Unmask (Enable) the interrupt.

Summary

- RIM → Reads the status of serial port and interrupt pins into Accumulator.
- SIM → Sets/controls interrupt masks and serial port operation from Accumulator.

Note :

- Both instructions do not affect any flag.
- Both are single byte instructions and take 1 machine cycle.